MarioSystems — Auth Gateway

Documentação Técnica Base

Informação	Definição
Projeto	Auth Gateway
Organização	MarioSystems
Categoria	Backend / Security
Linguagem	TypeScript
Framework	NestJS
Banco de dados	PostgreSQL
ORM	Prisma
Cache e gerenciamento de sessão	Redis
Conteinerização	Docker
Arquitetura	Hexagonal (Ports and Adapters)
Metodologia	TDD + BDD
Status	Em desenvolvimento

---

1. Visão Geral

1.1. Sobre o projeto

O Auth Gateway é um serviço backend especializado em autenticação e proteção de acesso, desenvolvido para centralizar os mecanismos de identificação de usuários, gerenciamento de sessões e validação de credenciais.

O sistema será construído utilizando NestJS e TypeScript, com PostgreSQL para persistência de dados, Prisma como ORM e Redis para operações que exigem armazenamento temporário e acesso rápido.

Sua arquitetura será organizada de maneira modular, seguindo os princípios da Arquitetura Hexagonal (Ports and Adapters), Clean Architecture e SOLID.

O projeto faz parte do ecossistema MarioSystems e será desenvolvido como um serviço independente, com APIs e contratos preparados para futuras integrações.

1.2. Problema que o sistema pretende resolver

Aplicações frequentemente implementam seus próprios mecanismos de autenticação, gerenciamento de sessões e validação de credenciais.

Essa abordagem pode resultar em:

- Duplicação de lógica de autenticação.
- Implementações inconsistentes de segurança.
- Dificuldade de gerenciamento de sessões.
- Falta de padronização entre aplicações.
- Dificuldade para auditar eventos de autenticação.
- Maior superfície de ataque devido a implementações independentes.

O Auth Gateway pretende oferecer uma solução centralizada e reutilizável para esses problemas.

1.3. Objetivo principal

Construir um serviço de autenticação seguro, modular, testável e extensível, capaz de atender diferentes aplicações por meio de uma API HTTP bem definida.

1.4. Objetivos técnicos

- Implementar autenticação baseada em credenciais.
- Gerenciar sessões e tokens de acesso.
- Implementar JWT e Refresh Tokens.
- Proteger recursos por meio de mecanismos de autorização.
- Aplicar RBAC (Role-Based Access Control).
- Implementar mecanismos de proteção contra abuso.
- Registrar eventos relevantes de segurança.
- Garantir confiabilidade por meio de testes automatizados.
- Aplicar princípios de arquitetura limpa e baixo acoplamento.
- Preparar o serviço para integração com outros sistemas.

---

2. Escopo e Responsabilidades

2.1. Responsabilidades do Auth Gateway

O sistema será responsável por:

1. Gerenciar contas de usuários.
2. Autenticar usuários por meio de credenciais.
3. Validar credenciais de autenticação.
4. Emitir tokens de acesso.
5. Gerenciar Refresh Tokens.
6. Renovar e revogar sessões.
7. Validar tokens recebidos pelas aplicações.
8. Aplicar mecanismos de autorização definidos para o serviço.
9. Proteger endpoints contra requisições abusivas.
10. Registrar eventos relevantes relacionados à autenticação e às sessões.

2.2. O que não faz parte do escopo inicial

Para evitar que o projeto cresça sem controle, algumas funcionalidades não farão parte da primeira versão:

- Gerenciamento corporativo completo de identidades.
- Administração avançada de políticas de acesso.
- Single Sign-On (SSO) entre provedores externos.
- Autenticação por redes sociais.
- Integração com LDAP ou Active Directory.
- Autenticação multifator (MFA).
- Biometria.
- Gerenciamento de organizações e múltiplos tenants.
- Sistema completo de detecção inteligente de ameaças.

Esses recursos poderão ser avaliados posteriormente, conforme a evolução do projeto.

O gerenciamento avançado de identidades, papéis e políticas será explorado principalmente pelo IAM Security System, outro projeto da MarioSystems.

2.3. Diferenciação entre Auth Gateway e API Gateway

O nome Auth Gateway não significa que o projeto será um proxy reverso ou um gateway responsável por rotear todas as requisições de uma infraestrutura.

Seu objetivo é fornecer serviços centralizados de autenticação e proteção de acesso.

O Nginx poderá ser utilizado posteriormente como proxy reverso na infraestrutura, mas essa responsabilidade é diferente da lógica de autenticação do Auth Gateway.

---

3. Tecnologias

3.1. Stack principal

Tecnologia	Responsabilidade
Node.js	Ambiente de execução
TypeScript	Linguagem principal
NestJS	Framework backend
PostgreSQL	Persistência relacional
Prisma	ORM e acesso ao banco
Redis	Armazenamento temporário e operações de sessão
Docker	Conteinerização
Jest	Testes automatizados
Supertest	Testes HTTP
Swagger / OpenAPI	Documentação da API

As ferramentas de testes, documentação e infraestrutura serão incorporadas conforme as etapas de implementação.

3.2. Responsabilidade de cada tecnologia

NestJS

Será responsável pela estrutura da aplicação, injeção de dependências, módulos, controllers, guards e integração com os serviços.

TypeScript

Será utilizado para garantir tipagem estática, contratos explícitos e maior previsibilidade durante o desenvolvimento.

PostgreSQL

Será utilizado para armazenar informações persistentes, como:

- Usuários.

- Credenciais criptograficamente protegidas.
- Papéis e permissões locais.
- Sessões e informações de Refresh Tokens.
- Dados necessários para auditoria.

Prisma

Será responsável pelo acesso tipado ao banco, consultas, relacionamentos e gerenciamento de migrations.

Redis

Será utilizado para operações temporárias e de acesso frequente, como:

- Controle de tentativas de autenticação.
- Rate Limiting distribuído.
- Dados temporários de sessão, quando necessário.
- Cache de informações de curta duração.

O Redis não será considerado a fonte definitiva dos dados de identidade ou das sessões persistentes.

Docker

Será utilizado para padronizar o ambiente de desenvolvimento, testes e implantação.

---

4. Arquitetura de Software

4.1. Arquitetura Hexagonal

O Auth Gateway seguirá o padrão Ports and Adapters.

A ideia central é manter as regras de negócio independentes de frameworks, bancos de dados e serviços externos.

O domínio não deve depender diretamente de NestJS, Prisma, Redis ou bibliotecas específicas de infraestrutura.

A comunicação entre as camadas ocorrerá por meio de interfaces e contratos.

Camadas principais

Domain

Contém as regras de negócio fundamentais.

Responsabilidades:

- Entidades.
- Objetos de valor.
- Regras e invariantes.
- Exceções de domínio.
- Contratos essenciais do domínio.

Application

Contém os casos de uso da aplicação.

Responsabilidades:

- Orquestração das operações.
- Regras de aplicação.
- Portas de entrada e saída.
- DTOs internos, quando aplicáveis.
- Coordenação entre domínio e infraestrutura.

Infrastructure

Contém implementações concretas das dependências externas.

Responsabilidades:

- Repositórios Prisma.
- Integração com PostgreSQL.
- Adaptadores Redis.
- Implementação de hashing.
- Implementação de JWT.
- Configurações de serviços externos.

Presentation

Contém os mecanismos de entrada e exposição da aplicação.

Responsabilidades:

- Controllers.
- DTOs HTTP.
- Validação de requisições.
- Guards.
- Interceptors.
- Filters.
- Serialização das respostas.

4.2. Direção das dependências

As dependências devem apontar para as regras internas da aplicação.

O domínio não conhece a infraestrutura.

A aplicação conhece os contratos necessários, mas não depende das implementações concretas.

A infraestrutura implementa os contratos definidos pelas camadas internas.

Fluxo conceitual

HTTP Request → Controller → Application Use Case → Domain → Port → Infrastructure Adapter → PostgreSQL / Redis

A resposta retorna pela cadeia de execução até o controller.

4.3. Organização modular

A aplicação será dividida por módulos de responsabilidade.

Estrutura conceitual:

```
None
src/
├── main.ts
├── app.module.ts
│
├── modules/
│   ├── users/
│   ├── authentication/
│   ├── sessions/
│   ├── authorization/
│   └── security/
│
├── shared/
│   ├── domain/
│   ├── application/
│   └── infrastructure/
│
└── config/
```

Cada módulo poderá conter suas próprias camadas:

```
None
users/
├── domain/
│   ├── entities/
│   ├── value-objects/
│   ├── repositories/
│   └── exceptions/
│
├── application/
│   ├── use-cases/
│   ├── ports/
│   └── dto/
│
├── infrastructure/
│   └── persistence/
│       ├── prisma/
│       └── repositories/
│
└── presentation/
    ├── controllers/
    └── dto/
```

Essa estrutura é uma referência arquitetural. Ela deverá ser adaptada ao código real, evitando criar pastas e abstrações sem necessidade.

---

5. Domínio e Modelagem

5.1. Entidades principais

O modelo inicial deverá considerar as seguintes entidades:

User

Representa uma conta cadastrada no sistema.

Atributos conceituais:

- id
- name
- email

- passwordHash
- status
- createdAt
- updatedAt

Responsabilidades:

- Representar a identidade local do usuário.
- Manter informações básicas da conta.
- Controlar o estado da conta.
- Garantir a unicidade do email.

O sistema não deverá armazenar senhas em texto puro.

Role

Representa um conjunto de responsabilidades e permissões associado a usuários.

Atributos conceituais:

- id
- name
- description
- createdAt
- updatedAt

Exemplos:

- USER
- ADMIN

Os papéis iniciais devem ser simples e suficientes para demonstrar o funcionamento do RBAC.

Permission

Representa uma ação autorizada dentro do sistema.

Atributos conceituais:

- id
- name
- resource
- action
- description

Exemplos:

- users
- users

- users
- users

UserRole

Representa o relacionamento entre usuários e papéis.

Permite associar mais de um papel a uma identidade, caso essa necessidade seja confirmada no modelo de negócio.

RolePermission

Representa o relacionamento entre papéis e permissões.

Permite determinar quais ações cada papel pode executar.

Session

Representa uma sessão autenticada.

Atributos conceituais:

- id
- userId
- refreshTokenHash
- expiresAt
- revokedAt
- createdAt
- lastUsedAt

A sessão será utilizada para controlar a renovação e revogação de tokens.

Os Refresh Tokens deverão ser armazenados de forma protegida, preferencialmente por meio de hash, nunca em texto puro no banco.

5.2. Relacionamentos conceituais

```
None
User
 ├── UserRole
 │     └── Role
 │           └── RolePermission
 │                 └── Permission
 │
 └── Session
```

5.3. Considerações sobre a modelagem

- Identificadores devem ser estáveis e não depender de informações pessoais.
- Email deverá possuir restrição de unicidade.
- Senhas serão armazenadas somente como hashes.
- Relacionamentos deverão possuir integridade referencial.
- Exclusões e alterações deverão respeitar as regras de negócio.
- Datas deverão ser armazenadas de maneira consistente, preferencialmente em UTC.
- Índices deverão ser definidos para campos frequentemente consultados.
- Migrations deverão ser versionadas.

A modelagem final deverá ser validada durante a implementação dos casos de uso.

---

6. Autenticação

6.1. Método de autenticação

A primeira versão utilizará autenticação por email e senha.

O fluxo deverá validar as credenciais, verificar o estado da conta e iniciar uma sessão autenticada.

6.2. Cadastro de usuário

Fluxo esperado:

1. Cliente envia nome, email e senha.
2. Controller valida o formato dos dados.
3. Caso de uso verifica a existência de uma conta com o mesmo email.
4. Serviço de hashing gera o hash da senha.
5. Repositório persiste o usuário.
6. Sistema retorna os dados públicos da conta.

A senha nunca deverá aparecer na resposta HTTP ou nos logs.

6.3. Login

Fluxo esperado:

1. Cliente envia email e senha.
2. Sistema aplica os controles de requisição.
3. Caso de uso localiza o usuário.

4. Serviço de hashing verifica a senha.
5. Sistema verifica se a conta está ativa.
6. Sistema cria uma sessão.
7. Sistema gera o Access Token.
8. Sistema gera o Refresh Token.
9. Cliente recebe as credenciais de autenticação.

O sistema deverá retornar mensagens genéricas para falhas de credenciais, evitando revelar se determinado email está cadastrado.

6.4. Hashing de senhas

A implementação deverá utilizar um algoritmo apropriado para armazenamento de senhas.

Recomendação inicial:

- Argon2id como primeira opção.
- Bcrypt como alternativa, caso exista uma justificativa técnica.

O algoritmo deverá utilizar parâmetros de custo adequados ao ambiente de execução.

O hash deverá ser armazenado no banco por meio do campo passwordHash.

Nunca utilizar criptografia reversível ou hash simples como SHA-256 para armazenar senhas.

---

7. JWT e Refresh Tokens

7.1. Access Token

O Access Token será um JWT utilizado para autenticar requisições protegidas.

Deverá conter somente informações necessárias para validar a identidade e o contexto de autorização.

Claims recomendadas:

- sub: identificador do usuário.
- iss: emissor.
- aud: destinatário.
- iat: data de emissão.
- exp: expiração.
- jti: identificador único do token.
- roles: papéis necessários à autorização, se aplicável.

Não incluir senhas, informações sensíveis ou dados desnecessários.

Expiração

Recomendação inicial: 15 minutos.

O tempo deverá ser configurável por ambiente.

7.2. Refresh Token

O Refresh Token será utilizado para obter novos Access Tokens sem exigir que o usuário informe suas credenciais novamente.

Características:

- Vida útil maior que o Access Token.
- Identificação associada a uma sessão.
- Armazenamento protegido.
- Possibilidade de revogação.
- Rotação a cada renovação bem-sucedida.

Recomendação inicial: 7 dias de validade, configuráveis.

7.3. Rotação de Refresh Tokens

A rotação deverá seguir este fluxo:

1. Cliente envia o Refresh Token atual.
2. Sistema valida a existência e a validade da sessão.
3. Sistema compara o token recebido com o hash armazenado.
4. Sistema invalida o token anterior.
5. Sistema gera um novo Refresh Token.
6. Sistema armazena o novo hash.
7. Sistema emite um novo Access Token.
8. Sistema devolve as novas credenciais.

A operação deverá ser atômica para evitar inconsistências.

7.4. Reutilização de tokens

Se um Refresh Token já utilizado for apresentado novamente, o sistema deverá tratar o evento como possível reutilização indevida.

Uma estratégia recomendada é revogar a família de tokens ou a sessão associada e registrar o evento de segurança.

7.5. Revogação

O sistema deverá permitir:

- Encerramento da sessão atual.
- Revogação de sessões específicas.
- Encerramento de todas as sessões do usuário.

A invalidação de um Refresh Token deverá impedir novas renovações.

Como JWTs de acesso são autocontidos, sua invalidação imediata exige uma estratégia adicional, como lista de revogação por jti no Redis ou validação de sessão. Essa decisão deverá equilibrar segurança, desempenho e complexidade.

7.6. Armazenamento no cliente

Para aplicações web, a estratégia preferencial será:

- Access Token mantido em memória.
- Refresh Token em cookie HttpOnly, Secure e SameSite.
- Proteção contra CSRF conforme o fluxo de cookies adotado.

Não utilizar localStorage como padrão para armazenar Refresh Tokens.

Clientes mobile poderão utilizar armazenamento seguro disponibilizado pelo sistema operacional.

---

8. Autorização e RBAC

8.1. Conceito

RBAC (Role-Based Access Control) é um modelo de controle de acesso baseado em papéis.

Em vez de definir permissões individualmente para cada usuário, o sistema associa permissões a papéis, que por sua vez são associados aos usuários.

8.2. Funcionamento

```
None
Usuário
   |
```

```
   v
Papéis associados
   |
   v
Permissões dos papéis
   |
   v
Decisão de autorização
   |
   v
Acesso permitido ou negado
```

8.3. Exemplos

Papel	Permissões
USER	Acessar os próprios dados
ADMIN	Gerenciar usuários e consultar informações administrativas

Esses papéis são exemplos iniciais, não uma definição definitiva de todos os privilégios.

8.4. Guards

O NestJS utilizará Guards para proteger rotas.

Responsabilidades:

- Validar a identidade autenticada.
- Verificar os papéis ou permissões exigidos.
- Impedir acesso não autorizado.
- Padronizar a proteção dos endpoints.

A autorização deverá ser aplicada no backend. Ocultar botões no frontend não constitui controle de acesso.

8.5. Princípio do menor privilégio

Toda conta deverá possuir somente as permissões necessárias para executar suas funções.

O sistema deverá negar acesso por padrão quando não existir uma regra explícita de autorização.

---

9. Redis e Gerenciamento de Sessões

9.1. Finalidade

O Redis será utilizado para operações temporárias e de baixa latência.

Possíveis utilizações:

- Rate Limiting.
- Contagem de tentativas de autenticação.
- Controle temporário de bloqueios.
- Lista de revogação de tokens.
- Cache de dados de autorização, se necessário.

9.2. Rate Limiting

O sistema deverá limitar requisições para evitar abuso dos endpoints.

Exemplos de aplicação:

- Login.
- Cadastro.
- Renovação de tokens.
- Recuperação de senha, caso seja adicionada posteriormente.

Os limites deverão ser configuráveis e aplicados de maneira distribuída.

O sistema deverá considerar tanto o endereço de origem quanto identificadores de conta, evitando que um atacante consiga contornar a proteção simplesmente alterando o IP.

9.3. Disponibilidade

O comportamento esperado quando Redis estiver indisponível deverá ser definido por funcionalidade.

Para controles críticos de segurança, não se deve simplesmente ignorar a proteção e liberar requisições sem limite.

A aplicação deverá registrar a falha e seguir a política de disponibilidade definida para cada operação.

---

10. Segurança da Aplicação

A segurança deverá ser considerada desde a concepção dos módulos.

10.1. Proteção de credenciais

- Nunca armazenar senhas em texto puro.
- Nunca registrar senhas em logs.
- Nunca retornar hashes de senhas pela API.
- Utilizar hashing apropriado.
- Proteger segredos por variáveis de ambiente ou gerenciador de segredos.

10.2. Proteção de tokens

- Validar assinatura.
- Validar expiração.
- Validar emissor e destinatário.
- Restringir algoritmos aceitos.
- Não confiar em claims sem validação.
- Utilizar chaves fortes.
- Definir estratégia de rotação de chaves.

10.3. Proteção HTTP

- Validação de payloads.
- Limitação de requisições.
- Configuração adequada de CORS.
- Helmet e cabeçalhos de segurança.
- Tratamento padronizado de erros.
- Proteção contra enumeração de usuários.
- Proteção contra força bruta.
- HTTPS obrigatório em ambientes públicos.

10.4. Segurança de configuração

- Não versionar arquivos .env com segredos.
- Manter .env.example sem credenciais reais.
- Utilizar segredos diferentes por ambiente.
- Não executar a aplicação com permissões desnecessárias.
- Manter dependências atualizadas.
- Evitar exposição de informações internas em erros.

10.5. Auditoria

Eventos relevantes deverão ser registrados, como:

- Login bem-sucedido.
- Falha de autenticação.
- Renovação de sessão.
- Revogação de sessão.
- Bloqueio temporário.
- Alteração de credenciais.
- Alterações de papéis e permissões.

Os registros não deverão armazenar senhas, tokens completos ou outros segredos.

---

11. API e Contratos HTTP

11.1. Convenções

A API deverá seguir padrões REST.

Prefixo recomendado:

```
None
/api/v1
```

Convenções:

- JSON como formato de comunicação.
- HTTP status codes apropriados.
- Nomes de recursos no plural.
- DTOs específicos para entrada e saída.
- Validação dos dados recebidos.
- Respostas de erro padronizadas.
- Versionamento da API.

11.2. Endpoints planejados

Usuários

Método	Endpoint	Responsabilidade
POST	/api/v1/users	Criar conta
GET	/api/v1/users/me	Consultar usuário autenticado

PATCH	/api/v1/users/me	Atualizar dados próprios

Autenticação

Método	Endpoint	Responsabilidade
POST	/api/v1/auth/login	Autenticar usuário
POST	/api/v1/auth/refresh	Renovar tokens
POST	/api/v1/auth/logout	Encerrar sessão atual
POST	/api/v1/auth/logout-all	Encerrar todas as sessões

Sessões

Método	Endpoint	Responsabilidade
GET	/api/v1/sessions	Listar sessões do usuário
DELETE	/api/v1/sessions/	Revogar sessão específica

Autorização

Método	Endpoint	Responsabilidade
GET	/api/v1/authorization/me	Consultar permissões efetivas do usuário

Esses endpoints representam o contrato inicial proposto. Os recursos administrativos de papéis e permissões deverão ser definidos conforme o escopo final do módulo de autorização.

11.3. Exemplos de requisição

Cadastro

```
None
{
  "name": "Maria Silva",
  "email": "maria@example.com",
  "password": "StrongPassword123!"
}
```

Login

```
None
{
  "email": "maria@example.com",
  "password": "StrongPassword123!"
}
```

Resposta de autenticação

Para uma estratégia baseada em cookies, o Refresh Token poderá ser enviado por Set-Cookie, enquanto o corpo da resposta contém:

```
None
{
  "accessToken": "jwt-access-token",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

O valor apresentado é ilustrativo. Tokens reais nunca deverão ser documentados com credenciais válidas.

11.4. Códigos HTTP

Código	Utilização
200	Operação concluída
201	Recurso criado
204	Operação concluída sem corpo
400	Requisição inválida
401	Falha de autenticação
403	Acesso não autorizado
404	Recurso não encontrado
409	Conflito de dados
422	Dados semanticamente inválidos, quando adotado

429	Limite de requisições excedido
500	Erro interno
503	Serviço temporariamente indisponível

11.5. Formato padronizado de erro

Exemplo:

```
None
{
  "statusCode": 401,
  "code": "INVALID_CREDENTIALS",
  "message": "Credenciais inválidas",
  "timestamp": "2026-11-05T20:00:00.000Z",
  "path": "/api/v1/auth/login"
}
```

As mensagens públicas deverão ser claras, mas não revelar informações internas, stack traces ou detalhes sensíveis.

---

12. Estratégia de Testes

12.1. TDD

O desenvolvimento seguirá Test-Driven Development nas regras críticas.

Ciclo:

1. Escrever um teste que descreve o comportamento esperado.
2. Executar o teste e confirmar sua falha.
3. Implementar o mínimo necessário.
4. Executar novamente.
5. Refatorar sem quebrar os comportamentos.

12.2. BDD

Os cenários de negócio serão descritos de maneira compreensível.

Exemplo:

```
None
Feature: Autenticação de usuários

Scenario: Login com credenciais válidas
  Given que existe um usuário ativo
  And que ele possui uma senha válida
  When ele envia suas credenciais
  Then o sistema deve autenticar o usuário
  And deve criar uma sessão
  And deve emitir um Access Token
```

12.3. Categorias de testes

Testes unitários

- Entidades.
- Objetos de valor.
- Casos de uso.
- Validadores.
- Serviços de domínio.
- Regras de autorização.

Testes de integração

- Repositórios Prisma.
- Persistência PostgreSQL.
- Operações Redis.
- Criação e revogação de sessões.
- Rotação de Refresh Tokens.

Testes end-to-end

- Cadastro.
- Login.
- Renovação.
- Logout.
- Proteção de rotas.
- RBAC.
- Rate Limiting.

12.4. Cenários críticos

A suíte deverá contemplar:

- Cadastro com email duplicado.
- Cadastro com senha inválida.
- Login com senha incorreta.
- Login com usuário inexistente.
- Login com conta desativada.
- Acesso sem token.
- Acesso com token expirado.
- Acesso com assinatura inválida.
- Acesso sem permissão.
- Refresh Token expirado.
- Refresh Token revogado.
- Reutilização de Refresh Token.
- Duas renovações concorrentes da mesma sessão.
- Excesso de tentativas de login.
- Indisponibilidade de Redis.
- Falha de persistência.

12.5. Cobertura

O foco será garantir testes fortes nos componentes críticos, especialmente:

- Hashing e verificação de senhas.
- Validação de tokens.
- Rotação e revogação de sessões.
- Guards.
- Regras de autorização.

A cobertura será acompanhada como indicador de qualidade, não como objetivo artificial de atingir 100% em todo o projeto.

---

13. Configuração e Variáveis de Ambiente

As configurações deverão ser externas ao código e validadas durante a inicialização.

Exemplo de referência:

```
None
NODE_ENV=development
PORT=3000
```

```
DATABASE_URL=postgresql://user:password@localhost:5432/auth_gateway

REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=

JWT_ACCESS_SECRET=
JWT_ACCESS_EXPIRES_IN=15m

JWT_REFRESH_SECRET=
JWT_REFRESH_EXPIRES_IN=7d

JWT_ISSUER=mariosystems-auth-gateway
JWT_AUDIENCE=mariosystems-services

FRONTEND_ORIGIN=http://localhost:5173
```

Essas variáveis representam uma configuração inicial proposta. A implementação deverá validar presença, formato e segurança dos valores.

Regras:

- Segredos não devem possuir valores padrão em produção.
- DATABASE_URL deverá ser protegida.
- Chaves JWT deverão possuir entropia adequada.
- Configurações deverão ser separadas por ambiente.
- Arquivos .env não deverão ser versionados.
- O projeto deverá disponibilizar um .env.example.

---

14. Infraestrutura e Docker

14.1. Serviços previstos

O ambiente conteinerizado deverá considerar:

- Auth Gateway.
- PostgreSQL.
- Redis.

O ambiente de desenvolvimento poderá utilizar Docker Compose para iniciar os serviços necessários.

14.2. Requisitos de infraestrutura

- Persistência de dados PostgreSQL por volume.
- Persistência Redis conforme necessidade operacional.
- Health checks.
- Rede interna entre containers.
- Segredos fora das imagens.
- Usuário sem privilégios desnecessários.
- Configuração reproduzível.
- Logs acessíveis.
- Reinicialização controlada.

14.3. Ambientes

Development

Ambiente local para desenvolvimento e testes manuais.

Testing

Ambiente isolado para execução de testes automatizados.

Production

Ambiente de implantação com configurações seguras, HTTPS, gerenciamento de segredos e monitoramento.

---

15. Observabilidade e Monitoramento

A aplicação deverá oferecer informações suficientes para identificar falhas e acompanhar seu comportamento.

15.1. Logs estruturados

Os logs deverão conter informações como:

- Timestamp.
- Nível do evento.
- Nome do serviço.
- Identificador de correlação.

- Operação executada.
- Resultado.
- Código de erro, quando aplicável.

Nunca registrar credenciais, tokens completos ou senhas.

15.2. Métricas

Métricas importantes:

- Quantidade de tentativas de login.
- Taxa de autenticações bem-sucedidas.
- Taxa de falhas de autenticação.
- Quantidade de sessões ativas.
- Quantidade de renovações.
- Quantidade de revogações.
- Requisições bloqueadas por Rate Limiting.
- Latência das operações.
- Disponibilidade do PostgreSQL e Redis.

15.3. Ferramentas previstas

- OpenTelemetry.
- Prometheus.
- Grafana.

A instrumentação deverá ser adicionada progressivamente, sem introduzir complexidade operacional desnecessária nas primeiras etapas.

---

16. Documentação e Integração

16.1. Swagger / OpenAPI

A API deverá possuir documentação interativa contendo:

- Endpoints.
- Métodos HTTP.
- Parâmetros.
- DTOs.
- Exemplos de requisição.
- Exemplos de resposta.
- Códigos de erro.
- Requisitos de autenticação.

16.2. Contratos de integração

O Auth Gateway deverá disponibilizar contratos estáveis para aplicações consumidoras.

Informações importantes:

- Como autenticar.
- Como validar Access Tokens.
- Como renovar sessões.
- Como revogar sessões.
- Como interpretar erros.
- Como consultar informações públicas do usuário autenticado.
- Como interpretar papéis e permissões.

A integração futura com o IAM Security System deverá respeitar a separação de responsabilidades e evitar dependências diretas entre os bancos de dados.

---

17. Estrutura de Desenvolvimento e Entregas

O desenvolvimento será dividido em etapas com objetivos verificáveis.

Fase 1 — Fundação

Objetivo: estabelecer a estrutura técnica do projeto.

- Setup inicial do NestJS.
- Configuração inicial do banco de dados.
- Modelagem inicial.
- Estrutura inicial do módulo de usuários.

Fase 2 — Gerenciamento de usuários

Objetivo: concluir as operações essenciais de contas.

- Criar usuário.
- Consultar usuário.
- Atualizar dados permitidos.
- Garantir unicidade de email.
- Implementar hashing de senha.
- Criar testes unitários e de integração.

Fase 3 — Autenticação

Objetivo: implementar o fluxo completo de autenticação.

- Login.
- Verificação de credenciais.
- Emissão de JWT.
- Criação de sessão.
- Refresh Tokens.
- Rotação de tokens.
- Logout.
- Revogação de sessões.

Fase 4 — Autorização

Objetivo: proteger recursos e operações.

- Modelagem de papéis.
- Modelagem de permissões.
- Relacionamentos entre usuários, papéis e permissões.
- Guards.
- Decorators.
- Testes de autorização.

Fase 5 — Segurança adicional

Objetivo: fortalecer o serviço contra abusos e ataques comuns.

- Rate Limiting.
- Proteção contra força bruta.
- Configuração de CORS.
- Cabeçalhos de segurança.
- Auditoria de eventos.
- Proteção contra reutilização de Refresh Tokens.

Fase 6 — Qualidade e infraestrutura

Objetivo: preparar o serviço para implantação.

- Testes end-to-end.
- Testes de segurança.
- Docker Compose.
- Health checks.
- Swagger.
- Logs estruturados.
- CI com GitHub Actions.

- Documentação final.
- Deploy de demonstração.

Fase 7 — Integração

Objetivo: preparar o Auth Gateway para trabalhar com os demais sistemas MarioSystems.

- Definir contratos de autenticação.
- Documentar claims e permissões.
- Definir estratégia de validação para serviços consumidores.
- Preparar integração com o IAM Security System.
- Preparar publicação de eventos de autenticação.

---

18. Critérios de Conclusão

O Auth Gateway será considerado funcional em sua primeira versão quando:

1. For possível cadastrar usuários com segurança.
2. O sistema autenticar usuários com credenciais válidas.
3. Senhas forem armazenadas somente em formato de hash.
4. Access Tokens forem emitidos e validados corretamente.
5. Refresh Tokens puderem ser rotacionados e revogados.
6. Sessões puderem ser consultadas e encerradas.
7. Rotas protegidas rejeitarem acessos não autenticados.
8. O RBAC impedir operações sem autorização.
9. O Rate Limiting funcionar conforme a política definida.
10. Os cenários críticos possuírem testes automatizados.
11. O ambiente puder ser executado de maneira reproduzível.
12. A API possuir documentação acessível.
13. Erros e eventos relevantes puderem ser rastreados.
14. Segredos e credenciais não forem expostos.

---

19. Decisões de Engenharia

As decisões técnicas deverão ser registradas para evitar mudanças arquiteturais sem justificativa.

Tema	Direção definida
Framework	NestJS

Linguagem	TypeScript
Banco	PostgreSQL
ORM	Prisma
Cache e operações temporárias	Redis
Arquitetura	Hexagonal
Organização	Modular
Autenticação	Email e senha
Tokens	JWT + Refresh Tokens
Autorização	RBAC
Testes	TDD + BDD
Infraestrutura	Docker
API	REST
Documentação	Swagger / OpenAPI

Decisões que precisam ser validadas durante a implementação

- Estratégia definitiva de armazenamento dos tokens no cliente.
- Algoritmo e parâmetros de hashing.
- Política de expiração e renovação de tokens.
- Estratégia de revogação imediata de Access Tokens.
- Modelo final de sessões.
- Limites de Rate Limiting.
- Estratégia de chaves JWT e rotação.
- Contratos de integração com o IAM.
- Política de auditoria e retenção dos eventos.

Toda mudança relevante deverá ter justificativa técnica e, quando necessário, registro de decisão arquitetural (ADR).

---

20. Princípios de Desenvolvimento

O desenvolvimento do Auth Gateway deverá seguir as seguintes diretrizes:

1. Não implementar funcionalidades sem compreender o problema que resolvem.
2. Não criar abstrações sem necessidade concreta.
3. Não misturar regras de negócio com detalhes do framework.
4. Não armazenar informações sensíveis sem proteção.
5. Não confiar em dados enviados pelo cliente.
6. Não considerar autenticação e autorização como a mesma responsabilidade.
7. Não permitir acesso por padrão quando não houver autorização.
8. Não tratar Redis como fonte definitiva de identidade.
9. Não implementar funcionalidades críticas sem testes.
10. Não considerar uma funcionalidade concluída apenas porque o endpoint responde.
11. Documentar decisões importantes.
12. Priorizar código que possa ser explicado, testado e mantido.

---

21. Referências Técnicas

Documentação oficial recomendada:

- NestJS: https://docs.nestjs.com/
- TypeScript: https://www.typescriptlang.org/docs/
- Prisma: https://www.prisma.io/docs
- PostgreSQL: https://www.postgresql.org/docs/
- Redis: https://redis.io/docs/latest/
- Docker: https://docs.docker.com/
- OWASP: https://owasp.org/
- OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/
- OWASP Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- JWT — RFC 7519: https://www.rfc-editor.org/rfc/rfc7519
- OAuth 2.0 Security Best Current Practice: https://www.rfc-editor.org/rfc/rfc9700
- OpenAPI: https://spec.openapis.org/oas/latest.html

MarioSystems — Auth Gateway

Building secure authentication systems through disciplined software engineering.